Android

Introdución **1**
Arquitectura da plataforma **1**
Sistema de arquivos de Android **3**
Particións de Android **4**
Xerarquía de arquivos en Android **4**
Fundamentos dunha aplicación Android **5**
Compoñentes 6
Activities 6
Servizos 6
Broadcast receivers 7
Intent 7
Content providers 8
AndroidManifest.xml 8
apk **9**
Seguridade en Android **10**
Servizos de seguridade de Android 11
Arquitectura de seguridade da plataforma 11
Seguridade de Linux 12
Sandbox de aplicacións 12
Permisos do sistema de arquivos 12
Linux con seguridade mellorada 13
Arranque verificado 13
Criptografía 13
Rooteo do dispositivo 13
Funcións de seguridade da persoa usuaria 14
Cifrado de almacenamento 14
Protección de credenciais na pantalla de bloqueo 14
Referencias **14**

Introdución

Android é un sistema operativo móbil baseado nunha versión modificada do núcleo de Linux, de código aberto, deseñado principalmente para dispositivos móbiles con pantalla táctil, como teléfonos intelixentes e tabletas.

Android foi desenvolvido historicamente por un consorcio de persoas desenvolvedoras coñecido como Open Handset Alliance, pero a súa versión máis utilizada está desenvolvida principalmente por Google.

Lanzado por primeira vez en 2008, Android é o sistema operativo máis utilizado do mundo. A última versión, lanzada o 10 de xuño de 2025, é Android 16. Esta versión, coñecida internamente como "Baklava", rompeu coa tradición dos nomes de sobremesa en orde alfabética e introduciu melloras centradas en intelixencia artificial xerativa e estabilidade do sistema.

En esencia, o sistema operativo coñécese como Android Open Source Project (AOSP) e é software libre e de código aberto licenciado principalmente baixo a Licencia Apache. Sen embargo, a maioría dos dispositivos executan a versión propietaria de Android desenvolvida por Google, que se envía con software propietario de código cerrado adicional preinstalado e inclúe aplicacións principais como Google Chrome, a plataforma de distribución dixital Google Play e a plataforma de desenvolvemento asociada Google Play Services.

Os paquetes de software en Android, que utilizan o formato APK, xeralmente distribúense a través de tendas de aplicacións propietarias como Google Play Store, Amazon Appstore, Samsung Galaxy Store, Huawei AppGallery, Cafe Bazaar, GetJar e Aptoide, ou plataformas de código aberto como F-Droid. Desde 2011, Android foi o sistema operativo máis utilizado en todo o mundo en teléfonos intelixentes.

Arquitectura da plataforma

Android é unha pila de software de código aberto baseado en Linux creada para unha variedade ampla de dispositivos e factores de forma. Pode consultarse a Platform architecture | Android Developers para máis información.

A arquitectura está formada por unha serie de capas apiladas, que se describen a continuación, empezando pola capa de máis baixo nivel:

- **Kernel de Linux**: é a base da plataforma de Android. Por exemplo, o Android Runtime (ART) baséase no kernel de Linux para funcionalidades subxacentes, como a xeración de subprocesos e a administración de memoria de baixo nivel.

  O uso dun kernel de Linux permite que Android aproveite funcións de seguridade clave e permite aos fabricantes de dispositivos desenvolver controladores de hardware para un kernel coñecido.

- **HAL (Hardware Abstraction Layer - Capa de Abstracción de Hardware)**: a capa de abstracción de hardware (HAL) fai de intermediaria entre o harware do dispositivo

e o sistema operativo. Permite que Android non teña que coñecer os detalles específicos do hardware.

Por exemplo, cando unha aplicación quere usar a cámara, Android chama á HAL da cámara, que traduce a petición ao hardware concreto. Isto fai que dous móbiles diferentes funcionen igual gracias á HAL, que é implementada polo fabricante.

- **Android runtime (ART):** é a contorna de execución das aplicacións de Android. As aplicacións Android están compiladas nun bytecode denominado DEX (DALVIK Executable), un formato de bytecode deseñado especificamente para Android e optimizado para un uso de memoria mínimo. ART encárgase de executar o bytecode, xestionar a memoria e optimizar o rendemento.

  Para os dispositivos con Android 5.0 (nivel de API 21) ou versións posteriores, cada app executa os seus propios procesos coa súa propia instancia de ART, é dicir está illada do resto de aplicacións.

  ART está deseñado para poder executar varias aplicacións de forma eficiente en dispositivos con pouca memoria executando arquivos de formato Dalvik Executable (DEX). Antes de Android 5.0 (nivel de API 21), Dalvik era a contorna de execución do sistema operativo.

  ART e o seu predecesor Dalvik creáronse orixinalmente especificamente para o proxecto Android.

- **Bibliotecas C/C++ nativas**: moitos compoñentes e servizos centrais do sistema Android, como ART e HAL, compílanse a partir de código nativo que require bibliotecas nativas escritas en C e C++.
- **Java API framework:** o conxunto completo de funcións do SO Android están dispoñibles a través de APIs escritas na linguaxe Java. Estas APIs forman os compoñentes básicos necesarios para crear apps para Android, simplificando a reutilización dos compoñentes e servizos básicos do sistema, como os seguintes:

  - Un sistema de visualización (view system) rico e extensible que pode utilizarse para construír a **interface de usuario** dunha aplicación, incluídas listas, cuadrículas, cadros de texto, botóns e incluso un navegador web integrable.
  - Un administrador de recursos que proporciona acceso a recursos que non son de código.
  - Un administrador de notificacións que permite que todas as aplicacións mostren alertas personalizadas na barra de estado.
  - Un administrador de actividades (**activity Manager**) que administra o ciclo de vida das aplicacións e proporciona unha pila de actividades de navegación común.
  - Provedores de contido (**content providers**) que permiten que as aplicacións accedan a datos doutras aplicacións, como a aplicación Contactos, ou que compartan os seus propios datos. As aplicacións poden crear *content providers* e expoñer os seus datos a outras aplicacións.

As persoas desenvolvedoras teñen acceso ás mesmas APIs que utiliza o sistema Android.

- **Aplicacións do sistema:** Android trae un conxunto de aplicacións básicas para correo electrónico, mensaxería SMS, calendarios, navegación por Internet, contactos e máis. As aplicacións incluídas coa plataforma non teñen un status especial entre as aplicacións que a persoa usuaria escolle instalar. Polo tanto, unha aplicación de terceiros pode converterse no navegador web predeterminado, a mensaxería SMS ou incluso o teclado predeterminado. Hai algunhas excepcións, como a aplicación de Configuración do sistema.

  As aplicacións do sistema funcionan como aplicacións para as persoas usuarias e tamén para proporcionar capacidades clave ás que as persoas desenvolvedoras poden acceder desde a súa propia aplicación. Por exemplo, se unha aplicación necesita enviar mensaxes SMS, non é necesario crear esa funcionalidade, pode invocarse directamente calquera aplicación SMS que xa estea instalada para enviar unha mensaxe á persoa destinataria especificada.

Sistema de arquivos de Android

Ao igual que outros sistemas operativos modernos, Android ten un sistema de arquivos baseado en discos. Hai dúas áreas de almacenamento: interna e externa. O almacenamento interno refírese á memoria incorporada, non volátil. Os dispositivos tamén poden vir con memoria extraíble (como unha tarxeta SD), o que se coñece como almacenamento externo.

Android soporta os seguintes sistemas de ficheiros:

- EXT2/EXT3/EXT4 (Extended File System): é o sistema de ficheiros estándar dos sistemas Linux. É capaz de manexar grandes volumes de datos e arquivos. Está optimizado para traballar con discos duros de grande capacidade e alta velocidade.
- F2FS (Flash Friendly File System): Introducido en 2012 por Samsung co kernel 3.8. Deseñouse para traballar con dispositivos de memoria flash. Ten rexistros estruturados para optimizar a memoria flash NAND.
- exFAT (Extended File Allocation Table) + VFAT: é unha extensión de FAT16 e FAT32. Soen usarse nas SD cards grandes, entre outros.
- JFFS2 (Journal Flash File System 2): é o sistema de ficheiros por defecto para os kernels dos proxectos Open Source de Android. Apareceu con Android Ice Cream Sandwich (4.0). Hoxe en día non se usa.
- YAFFS2 (Yet Another Flash File System): deseñado para ser rápido co uso da memoria flash NAND. Non ten soporte a partir de Gingerbread, cando se pasou a EXT4.
- RFS (Robust File System): foi un sistema de ficheiros Android utilizado en versións antigas de Android, principalmente en dispositivos Samsung. Utilizouse por primeira vez no teléfono Samsung Galaxy S.

Particións de Android

Os dispositivos Android inclúen varias particións que cumpren diferentes funcións no proceso de arranque. Entre as partición que ten un dispositivo Android están as seguintes:

- **boot**: partición de arranque do sistema. Contén unha imaxe do kernel.
- **system**: partición do sistema operativo. Contén os arquivos do sistema operativo e as aplicacións do sistema que están preinstaladas. Esta partición está configurada como de só lectura.
- **recovery:** partición de recuperación. Alternativa á partición boot que permite ao dispositivo iniciar en modo “recovery”.
- **userdata**: partición que contén os datos da persoa usuaria, como as aplicacións instaladas, contactos, mensaxes, etc.
- **cache:** para almacenar datos e compoñentes que son accedidos con frecuencia.
- **misc:** almacena datos de configuración interna do dispositivo.
- **efs:** contén información relevante e crítica para o correcto funcionamento do dispositivo, como, por exemplo, a identificación do dispositivo (IMEI, número de serie, dirección MAC de rede, …). O nome EFS vén de Encrypted File System (Sistema de arquivos encriptado), que é un sistema de arquivos seguro que usan algúns dispositivos Android para almacenar esta información.

Xerarquía de arquivos en Android

A xerarquía de arquivos en Android refírese á estrutura de carpetas e arquivos que conforman o sistema de arquivos do dispositivo. Esta xerarquía de arquivos esténdese dende a raíz do sistema ata as carpetas de aplicacións e os arquivos de cada aplicación.

A xerarquía de ficheiros en Android é similar á de outros sistemas operativos baseados en Unix, como Linux.

Ademais das carpetas principais, tamén hai outras carpetas que conteñen arquivos e recursos específicos de cada aplicación instalada no dispositivo.

Un sistema Android ten, entre outras, as seguintes carpetas:

- **/boot:** contén os arquivos necesarios para que o sistema arranque de forma correcta.
- **/system:** contén arquivos e carpetas fundamentais do sistema operativo, necesarios para que o dispositivo funcione correctamente. En Android este directorio contén arquivos do sistema, bibliotecas, binarios e aplicacións do sistema que se instalan de forma predeterminada no dispositivo. Normalmente é de **só lectura**.
- **/recovery:** contén arquivos relacionados co proceso de recuperación do sistema operativo.
- **/data:** contén datos da persoa usuaria e das aplicacións instaladas. Contén arquivos con datos das aplicacións instaladas, arquivos de usuario, arquivos temporais e arquivos de sistema.
- **/cache:** almacena arquivos temporais que se utilizan para mellorar o rendemento do sistema e das aplicacións.

Fundamentos dunha aplicación Android

As aplicacións de Android poden escribirse con Kotlin, Java ou C++. As ferramentas do SDK de Android compilan o código, xunto cos arquivos de datos e recursos nun APK ou un Android App Bundle.

Un paquete de Android, que é un arquivo con extensión .**apk**, contén o contido dunha aplicación de Android e é o arquivo que os dispositivos con Android utilizan para instalar a aplicación.

Un Android App Bundle é o formato moderno de publicación de aplicacións. É un arquivo de almacenamento con extensión .**aab** que contén o contido dun proxecto de aplicación Android. Un AAB é un formato de publicación e non se pode instalar directamente en dispositivos Android. Súbese o arquivo .aab a Google Play e posteriormente xerarase o APK adaptado a cada dispositivo.

Cada aplicación Android execútase no seu propio sandbox, protexida polas seguintes características.

- O sistema operativo Android é un sistema Linux multiusuario no que cada aplicación é un usuario diferente.
- De forma predeterminada, o sistema asigna a cada aplicación un ID de usuario de Linux único, que só utiliza o sistema e que a aplicación descoñece. O sistema establece permisos para todos os arquivos dunha aplicación de modo que só o ID de usuario asignado a esa aplicación poida acceder a eles.
- Cada proceso ten a súa propia máquina virtual (VM), polo que o código de cada aplicación execútase de forma illada doutras aplicacións.
- De forma predeterminada, cada aplicación execútase no seu propio proceso Linux. O sistema Android inicia o proceso cando é necesario executar algún dos compoñentes da aplicación e despois deteno cando xa non é necesario ou cando o sistema debe recuperar memoria para outras aplicacións.

O sistema Android implementa o **principio de mínimo privilexio**. É dicir, cada aplicación, por defecto, ten acceso unicamente aos compoñentes que necesita para realizar o seu traballo e nada máis. Isto crea unha contorna segura na que unha aplicación non pode acceder a partes do sistema para as que non ten permiso.

Tamén existen formas nas que unha aplicación pode compartir datos con outras e acceder aos servizos do sistema:

- É posible organizar que dúas aplicacións compartan o mesmo ID de usuario de Linux, polo que poderán acceder aos arquivos da outra. Para conservar os recursos do sistema, as aplicacións co mesmo ID de usuario tamén poden organizarse para executarse no mesmo proceso de Linux e compartir a mesma máquina virtual. As aplicacións tamén deben estar firmadas co mesmo certificado.
- Unha aplicación pode solicitar permiso para acceder a datos do dispositivo, como a localización, a cámara e a conexión Bluetooth. A persoa usuaria debe conceder eses permisos de forma explícita.

Compoñentes

Os compoñentes principais dunha aplicación Android son os elementos básicos para construír a aplicación. Cada compoñente é un punto de entrada a través do cal o sistema, outras aplicacións ou unha persoa poden ingresar na aplicación.

Hai catro tipos de compoñentes de aplicación:

- Actividades (Activities).
- Servizos
- Broadcast receivers
- Provedores de contido (Content providers)

Cada tipo cumpre unha función específica e ten un ciclo de vida determinado que define como se crea e se destrúe un compoñente.

Activities

Unha activity é o punto de entrada para interactuar coa aplicación. Representa unha única **pantalla** con unha interface de usuario. Por exemplo, unha aplicación de correo electrónico pode ter unha activity que mostre unha lista de correos electrónicos novos, outra activity para redactar un correo electrónico e outra activity para ler correos electrónicos. Aínda que as activities funcionan xuntas para formar unha experiencia de usuario coherente na aplicación de correo electrónico, cada unha é independente das demais.

Unha aplicación diferente pode iniciar calquera das activities do correo electrónico se a aplicación de correo o permite. Por exemplo, unha aplicación de cámara podería iniciar a actividade na aplicación de correo electrónico para redactar un novo correo electrónico e permitir que a persoa usuaria comparta unha imaxe.

Unha aplicación a miúdo consta de varias activities relacionadas unhas con outras:

- A activity principal é a que se mostra ao usuario cando se lanza a aplicación.
- Unhas activities poden lanzar outras
- O sistema almacena as activities nunha pila, polo que a última en entrar é a primeira en saír.

Servizos

Un servizo é un compoñente que se usa para executar tarefas en **segundo plano**, sen interface de usuario. É un compoñente que se executa en segundo plano para realizar operacións de larga duración ou traballo de procesos remotos sen necesidade dunha interface gráfica.

Por exemplo, un servizo podería reproducir música en segundo plano mentres a persoa usuaria utiliza outra aplicación. Outros exemplos de servozos son a sincronización de datos, a descarga ou subida arquivos, etc.

Outro compoñente da aplicación, como unha activity, pode iniciar un servizo.

Broadcast receivers

Un broadcast receiver é un compoñente que serve para **recibir e responder a mensaxes (broadcasts) do sistema ou doutras aplicacións.** Debido a que os broadcast receivers son outra entrada ben definida na aplicación, o sistema pode enviar mensaxes incluso a aplicacións que non se estean executando actualmente.

Un Broadcast Receiver pode verse como un **listener** que atende a eventos do sistema, é dicir, lanzados polo sistema operativo, ou ben a eventos lanzados por aplicacións. O evento pode incluír información adicional nun Intent.

É un mecanismo que permite integrar aplicacións entre si e/ou con Android, pero tamén establecer un canal de comunicación interno entre compoñentes dunha mesma app.

Moitos broadcasts orixínanse no sistema, como unha mensaxe que anuncia que a batería está baixa, un cambio de rede (WiFi/móbil), chegada de SMS, conexión/desconexión do cargador, etc. As aplicacións tamén poden iniciar mensaxes para informar a outras aplicacións, por exemplo, de que se descargaron algúns datos ao dispositivo e que están dispoñibles para que os usen.

Cando un Broadcast receiver recibe un evento pode realizar varias tarefas: mostrar unha notificación, iniciar un Service, executar unha acción na aplicación, etc.

Intent

Un **Intent** é un mecanismo de comunicación que permite que os distintos compoñentes dunha app (ou entre apps) **se envíen mensaxes para realizar accións**. É dicir, un Intent representa unha mensaxe entre compoñentes. O intent conterá información coa acción a realizar e información adicional necesaria para acción.

Un aspecto único do deseño do sistema Android é que calquera aplicación pode iniciar o compoñente doutra aplicación. Por exemplo, para sacar unha foto non é necesario programar esta acción nunha aplicación, pode iniciarse esta actividade na aplicación da cámara do teléfono. Cando se saque a foto, esta pode ser enviada á aplicación que a solicitou.

Debido a que o sistema executa cada aplicación nun proceso independente con permisos de arquivo que restrinxen o acceso a outras aplicacións, unha aplicación non pode activar directamente un compoñente doutra aplicación. Sen embargo, o sistema Android si que pode facelo. Para activar un compoñente noutra aplicación, envíase unha mensaxe ao sistema que especifica a intención (**intent**) de iniciar un compoñente en particular. Despois, o sistema activará o compoñente.

Os Intents son usados para intercambiar información entre compoñentes e dita información pode ser con fins soamente comunicativos ou tamén para executar determinadas accións preestablecidas.

Content providers

Un **content provider** ou provedor de contido é un compoñente que permite **compartir e xestionar datos entre aplicacións de forma segura**. A través do provedor de contido, outras aplicacións poden consultar ou modificar os datos, se o provedor de contido o permite. É dicir, controla o acceso aos datos mediante permisos evitando o acceso inseguro.

Podería dicirse que un Content provider é unha “porta de acceso” aos datos dunha aplicación para que outras aplicacións poidan lelos ou modificalos de forma controlada. Por exemplo, cando se crea unha aplicación cunha base de datos, esta é privada por defecto e só accesible pola propia aplicación. Utilizando un Content Provider podería facerse que os datos estivesen visibles para o resto de aplicacións.

O sistema operativo Android incorpora de serie moitas aplicacións con Content Providers como poden ser os Contactos, Calendario, Mensaxes, … O resto de aplicacións poderán acceder aos contactos, por exemplo, usando un content provider.

AndroidManifest.xml

Antes de que o sistema Android poida iniciar un compoñente dunha aplicación, debe saber que o compoñente existe lendo o arquivo AndroidManifest.xml. A aplicación declara os seus compoñentes neste arquivo, que está na raíz do directorio do proxecto.

Ademais, no ficheiro AndroidManifest.xml especifícanse outras configuracións da aplicación coma as seguintes:

- Os compoñentes da aplicación, incluídas todas as **activities, services, broadcast receivers** e **content providers.** Cada compoñente debe definir propiedades básicas, como o nome da súa clase.
- Os permisos que necesita a aplicación para acceder a partes protexidas do sistema ou a outras aplicacións. Tamén declara os permisos que deben ter outras aplicacións se queren acceder ao contido desta aplicación.
- Declara o nivel de API mínimo requirido pola aplicación, en función das API que utiliza.
- Declara as características de hardware e software utilizadas ou requiridas pola aplicación, como unha cámara, servizos Bluetooth ou unha pantalla multitáctil.
- Declara as bibliotecas API coas que a aplicación debe vincularse (ademais das API do framework de Android).

No ficheiro AndroidMannifest.xml tamén se informa sobre os compoñentes da aplicación: activities, services, receivers e providers. Por exemplo, pode declararse unha actividade da seguinte maneira:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest ... >
   <application android:icon="@drawable/app_icon.png" ... >
      <activity android:name="com.example.project.ExampleActivity"
            android:label="@string/example_label" ... >
      </activity>
      ...
   </application>
</manifest>
```

Cando se declara unha actividade, poden incluírse filtros de intents que declaren as capacidades da actividade para responder a intents doutras aplicacións. O seguinte exemplo mostra o arquivo de manifesto dunha aplicación de correo electrónico con unha actividade para redactar un novo correo electrónico que declara un intent para enviar un novo correo electrónico:

```
<manifest ... >
   ...
   <application ... >
      <activity android:name="com.example.project.ComposeEmailActivity">
         <intent-filter>
            <action android:name="android.intent.action.SEND" />
            <data android:type="*/*" />
            <category android:name="android.intent.category.DEFAULT" />
         </intent-filter>
      </activity>
   </application>
</manifest>
```

As activities, servizos e content providers que non se declaran no arquivo AndroidManifest.xml non son visibles para o sistema e, en consecuencia, nunca poden executarse. Sen embargo, os **Broadcast receivers** poden declararse no manifesto ou crearse dinamicamente no código como un obxecto BroadcastReceiver e rexistrarse no sistema mediante unha chamada a registerReceiver().

apk

APK (Android Package) é o formato de arquivo utilizado para distribuír e instalar aplicacións móbiles no sistema operativo Android. Contén todos os recursos relacionados con unha aplicación. É unha variante do formato JAR de Java.

Un APK é basicamente un arquivo que contén os seguintes arquivos:

- carpeta **META-INF**:
    - MANIFEST.MF – Un arquivo de manifesto que contén hashes SHA de todos os arquivos do paquete APK;
    - Certificado da aplicación
    - Arquivos con metadatos adicionais.
- **lib**: carpeta que contén bibliotecas compiladas dependentes da plataforma. Inclúe subdirectorios para as plataformas correspondentes: armeabi, armeabi-v7a, arm64-v8a, x86, x86_64, mips.
- **res**: carpeta que contén recursos de Android: gráficos, deseños de interface, elementos multimedia, imaxes, etc.
- **assets**: carpeta que contén recursos extra utilizados pola aplicación: fontes, arquivos multimedia, etc.
- **AndroidManifest.xml**: arquivo de manifesto da aplicación Android. Este arquivo almacénase en formato XML binario compilado.
- **classes.dex**: un ou máis arquivos que representan o código da aplicación compilado en bytecode Dalvik.
- **resources.arsc**: arquivo que contén unha táboa de mapeo dos identificadores de recursos aos seus arquivos correspondentes.

Seguridade en Android

Android incorpora funcións de seguridade e traballa para manter segura a plataforma e o ecosistema de Android. Un modelo de seguridade sólido é esencial para permitir un sistema robusto de aplicacións e dispositivos.

Android é de código aberto, mais ofrece unha contorna que protexe a confidencialidade, integridade, dispoñibilidade de persoas usuarias, datos, aplicacións, dispositivo e rede.

Para protexer unha plataforma aberta necesítase unha arquitectura de seguridade sólida e programas de seguridade rigorosos. O sistema de seguridad de Android foi deseñado en base a varias **capas**, de forma que é o suficientemente flexible como para admitir que a plataforma sexa aberta e, ao mesmo tempo, protexe a todas as persoas usuarias da mesma.

A nivel de sistema operativo, Android proporciona a seguridade do núcleo Linux, así como unha función de comunicación segura entre procesos (IPC inter-process communication) que permite a comunicación segura entre as aplicacións, que se executan en diferentes procesos. Estas características de seguridade a nivel de sistema operativo fan que incluso o código nativo se execute na súa propia Sandbox. É dicir, o sistema está deseñado para evitar que unha aplicación maliciosa dane outras aplicacións, o sistema ou o dispositivo en si.

Servizos de seguridade de Android

Google ofrece un conxunto de servizos baseados na nube que están dispoñibles para dispositivos Android. Aínda que estes servizos non son parte do Android Open Source Project (AOSP), están incluídos en moitos dispositivos.

Os principais servizos de seguridade son:

- **Google Play**: Google Play é unha colección de servizos que permiten ás persoas usuarias descubrir, instalar e comprar aplicacións desde o seu dispositivo Android ou a Web. Google Play facilita ás persoas desenvolvedoras chegar ás persoas usuarias de Android e aos clientes potenciais. Google Play tamén ofrece revisión da comunidade, verificación de licenzas de aplicacións, análise de seguridade de aplicacións e outros servizos de seguridade.
- **Actualizacións de Android**: o servizo de actualización de Android ofrece novas capacidades e actualizacións de seguridade a certos dispositivos Android.
- **Servizos de aplicacións**: frameworks que permiten que as aplicacións de Android utilicen capacidades da nube, como facer copias de seguridade de datos e configuracións de aplicacións.
- **Verificar aplicacións**: advirte ou bloquea automaticamente a instalación de aplicacións maliciosas e escanea continuamente as aplicacións no dispositivo, advertindo sobre aplicacións maliciosas ou eliminándoas.
- **SafetyNet**: un sistema de detección de intrusións que preserva a privacidade para axudar ao seguimento de Google, mitigar ameazas de seguridade coñecidas e identificar novas ameazas de seguridade.
- **Administrador de dispositivos Android**: unha aplicación web e unha aplicación de Android para localizar dispositivos perdidos ou roubados.

Arquitectura de seguridade da plataforma

Android busca ser o sistema operativo máis seguro e utilizable para plataformas móbiles ao reutilizar os controis de seguridade do sistema operativo tradicional para:

- Protexer os datos da aplicación e da persoa usuaria.
- Protexer os recursos do sistema (incluída a rede)
- Proporcionar illamento de aplicacións con respecto ao sistema, das outras aplicacións e da persoa usuaria.

Para lograr estes obxectivos, Android proporciona estas características de seguridade clave:

- Seguridade robusta a nivel de SO a través do kernel Linux
- Sandbox obrigatorio para todas as aplicacións
- Comunicación segura entre procesos
- Firma de aplicacións
- Permisos definidos pola aplicación e outorgados pola persoa usuaria

Por seguridade, Android non permite instalar ningunha aplicación que non estea asinada, o que permite coñecer a súa procedencia. En Android, pode usarse un certificado autoasinado para asinar a aplicación.

Seguridade de Linux

A base da plataforma Android é o núcleo de Linux. O núcleo Linux utilízase en millóns de contornas sensibles á seguridade. Ao longo da súa historia Linux converteuse nun núcleo estable e seguro no que confían moitas corporacións e profesionais da seguridade.

O kernel de Linux proporciona a Android varias funcións de seguridade clave, entre as que se inclúen:

- Un modelo de permisos baseado na persoa usuaria
- Illamento de procesos
- Mecanismo extensible para IPC seguro (inter-process communication)
- Capacidade de eliminar partes innecesarias e potencialmente inseguras do kernel

Como sistema operativo multiusuario, un obxectivo de seguridade fundamental do núcleo de Linux é illar os recursos das persoas usuarias entre si. A filosofía de seguridade de Linux é protexer os recursos das persoas usuarias:

- Impide que o usuario A lea os arquivos do usuario B
- Asegura que o usuario A non esgote a memoria do usuario B
- Asegura que o usuario A non esgote os recursos de CPU do usuario B
- Asegura que o usuario A non esgote os dispositivos do usuario B (por exemplo, telefonía, GPS e Bluetooth)

Sandbox de aplicacións

A seguridade das aplicacións de Android vese reforzada polo Sandbox de aplicacións, que illa as aplicacións entre si e protexe as aplicacións e o sistema de aplicacións maliciosas.

Salvo unha pequena parte do sistema operativo que se executa como root, todo o que se executa por encima do kernel de Linux, execútase nunha sandbox.

A interacción entre aplicacións sempre se fai por mediación do sistema operativo.

O acceso aos recursos está protexido por un sistema de permisos que as aplicacións deben solicitar: cámara, teléfono, contactos, envío e recepción de SMS, acceso a Internet, acceso a outros medios de comunicación (Bluetooth, NFC, etc).

Para obter máis información, consulte Sandbox de aplicacións.

Permisos do sistema de arquivos

Nunha contorna de estilo UNIX, os permisos do sistema de arquivos garanten que unha persoa non poida modificar nin ler arquivos doutra persoa.

No caso de Android, cada aplicación execútase como un usuario diferente. Cando se instala unha aplicación, Android asígnalle un UID e un GID únicos. As aplicacións execútanse na súa propia Sandbox co seu propio proceso e UID.

Ademais, cada aplicación ten o seu propio directorio privado de datos, só accesible polo UID especificado.

A menos que a persoa desenvolvedora comparta arquivos explicitamente con outras aplicacións, os arquivos creados por unha aplicación non poden ser lidos nin modificados por outra aplicación.

Linux con seguridade mellorada

Android utiliza Security-Enhanced Linux (SELinux) para aplicar políticas de control de acceso e establecer un control de acceso obrigatorio (mac - mandatory access control) nos procesos.

SELinux utiliza o principio de denegación de permisos por defecto, é dicir, denega todo que non estea explicitamente permitido.

Consulta Security-Enhanced Linux en Android para obter máis información.

Arranque verificado

Android 7.0 e as versións posteriores admiten o arranque verificado de forma estrita, o que significa que dispositivos que están comprometidos non poden arrancar. O arranque verificado garante a integridade do software do dispositivo dende o hardware ata a partición do sistema. Durante o arranque, cada etapa verifica criptograficamente a integridade e autenticidade da seguinte etapa antes de executala. Consulta Arranque verificado para obter máis detalles.

Criptografía

Android ofrece un conxunto de APIs criptográficas para que as utilicen as aplicacións. Estas inclúen implementacións de primitivas criptográficas estándar e de uso común, como AES, RSA, DSA e SHA. Ademais, proporciónanse APIs para protocolos de nivel superior, como SSL e HTTPS.

Android 4.0 introduciu a clase KeyChain para permitir que as aplicacións utilicen o almacenamento de credenciais do sistema para claves privadas e cadeas de certificados.

Rooteo do dispositivo

De forma predeterminada, en Android só o núcleo e un pequeno subconxunto dos servizos principais se executan con permisos de root. O arranque verificado impide que unha persoa usuaria ou servizo con permisos de root modifique permanentemente o sistema operativo.

Para as persoas desenvolvedoras é importante poder modificar un dispositivo Android durante o seu traballo. En moitos dispositivos Android, as persoas usuarias teñen a capacidade de desbloquear o cargador de arranque para permitir a instalación dun sistema operativo alternativo. Estes sistemas operativos alternativos poden permitir que se obteña acceso root para depurar aplicacións e compoñentes do sistema ou para acceder a funcións non dispoñibles nas API de Android.

Funcións de seguridade da persoa usuaria

Cifrado de almacenamento

A CDD (Android Compatibility Definition Document - CDD) require que todos os dispositivos que se lanzan con Android 10 ou superior, e a maioría dos dispositivos que se lanzan con Android 6.0 ou superior, habiliten o cifrado de almacenamento de fábrica. É dicir, o almacenamento interno está cifrado por defecto, o que impide o acceso aos datos do dispositivo se este está bloqueado.

A actual implementación do cifrado de almacenamento de Android é o cifrado baseado en arquivos en combinación co cifrado de metadatos. O cifrado baseado en arquivos cifra de forma transparente o contido e os nomes dos arquivos na partición de datos de usuario, utilizando claves diferentes para distintos directorios. Proporciona directorios de almacenamento cifrados por credenciais e por dispositivo para cada usuario.

O cifrado de metadatos complementa o cifrado baseado en arquivos. Cifra todos os bloques da partición de datos de usuario que non estean cifrados mediante cifrado baseado en arquivos, utilizando unha clave que non está vinculada a ningunha credencial da pantalla de bloqueo da persoa usuaria, pero que está protexida polo arranque verificado.

Protección de credenciais na pantalla de bloqueo

Android pode configurarse para verificar unha credencial de pantalla de bloqueo proporcionada pola persoa usuaria (PIN, contrasinal ou patrón) antes de proporcionar acceso a un dispositivo. Ademais de evitar o uso non autorizado do dispositivo, a credencial de pantalla de bloqueo protexe a clave criptográfica para os datos cifrados con credenciais. O administrador do dispositivo pode esixir o uso dunha credencial de pantalla de bloqueo ou regras de complexidade de credenciais.

Referencias

Para a elaboración deste material utilizáronse, entre outros, os recursos que se enumeran a continuación:

- Curso F2101037 Posta en produción segura
- INCIBE
- Fernández Riera, Máximo (2022) Puesta en producción segura. Ra-Ma.
- Romero, Ternero, Mª Carmen (2024) Puesta en producción segura. Paraninfo.
- Platform architecture | Android Developers
- File System Of Android. The Android OS is a popular and… | by Aditi Kale | Medium
- Sistema de ficheros Android [Guía 2025] | KeepCoding
- Jerarquía de ficheros en Android | KeepCoding Bootcamps
- Application fundamentals | App architecture | Android Developers
- What Is an APK File? Meaning, Components, Operations, and Applications
- Secure an Android device